

Atividade 20 — Node.js, Express, EJS e Módulos de Rotas

Site da Semana de Tecnologia — sem banco de dados — baseado na apostila até o item 7.5.3.2

Duração da atividade:	Durante a aula
Modalidade:	Em dupla mas cada um deve subir como Atividade20
Pré-requisitos:	Node.js e NPM instalados; apostila até o item 7.5.3.2

1. Contexto

Você foi contratado(a) para criar o site institucional (estático, sem banco de dados) da “Semana de Tecnologia” da sua instituição de ensino, um evento com palestras e workshops sobre tecnologia. O site deve ser construído em Node.js, utilizando Express e EJS, seguindo o mesmo padrão de organização visto na apostila: servidor configurado em um módulo separado e cada rota implementada em seu próprio arquivo dentro de uma pasta routes.

2. O que já pode ser usado (visto na apostila)

- Criação de servidor com Express (app.get, res.send, res.render);
- Configuração do EJS como view engine (app.set);
- Módulos e CommonJS (require / module.exports);
- Nodemon para reiniciar o servidor automaticamente;
- Organização da aplicação em pastas (views, config, routes).

3. Estrutura de pastas

Crie a estrutura de pastas exatamente como no exemplo abaixo:

```
semana-tecnologia/  
├── app.js  
├── package.json  
├── app/  
│   ├── config/  
│   │   └── server.js  
│   ├── views/  
│   │   ├── home/  
│   │   │   └── index.ejs  
│   │   ├── evento/  
│   │   │   ├── sobre.ejs  
│   │   │   ├── programacao.ejs  
│   │   │   └── contato.ejs  
│   │   └── admin/  
│   │       └── inscricao.ejs  
└── routes/  
    ├── home.js  
    ├── sobre.js  
    ├── programacao.js  
    ├── contato.js  
    └── inscricao.js
```

4. Passo a passo sugerido

1. Criar a pasta do projeto e rodar npm init (ou npm init -y) para gerar o package.json.

2. Instalar as dependências: `npm install express --save`, `npm install ejs --save` e `npm install nodemon --save-dev`.
3. Ajustar o script "start" do package.json para usar o nodemon ("start": "npx nodemon app.js").
4. Criar o módulo app/config/server.js: nele, o Express é instanciado, o view engine é configurado como ejs, o caminho das views é configurado (`app.set('views', './app/views')`) e o app é exportado com `module.exports`.
5. Criar as 5 views .ejs indicadas na tabela da seção 6, com HTML simples (pode reaproveitar a estrutura básica de HTML vista na apostila).
6. Criar os 5 arquivos de rota dentro de app/routes/. Cada arquivo deve exportar uma `function(app)` que define a(s) rota(s) daquele módulo, exatamente como no padrão da apostila (item 7.5.3.2).
7. No app.js: dar require no módulo do servidor (app/config/server), dar require em cada módulo de rota e executá-lo passando o app como parâmetro (ex.: `rotaHome(app);`). Por fim, chamar `app.listen` na porta 3000.
8. Rodar `npm start` e testar cada uma das rotas no navegador, conferindo se todas retornam a página correta.

5. Rotas que a aplicação deve ter

Rota (URL)	Arquivo de rota	View (.ejs)	Conteúdo esperado
/	home.js	views/home/index.ejs	Página inicial do evento, com o nome da Semana de Tecnologia e uma frase de apresentação
/sobre	sobre.js	views/evento/sobre.ejs	Texto sobre o objetivo do evento, data(s) e local (pode ser inventado)
/programacao	programacao.js	views/evento/programacao.ejs	Lista (tags) com pelo menos 6 palestras/workshops e seus horários
/contato	contato.js	views/evento/contato.ejs	E-mail, telefone e local de informações da organização do evento
/admin/inscricao	inscricao.js	views/admin/inscricao.ejs	Página estática simulando uma futura tela de inscrição (formulário que ainda não grava dados)

7. Desafio extra (para quem terminar antes do tempo)

- Criar uma sexta rota, /novidades, com seu respectivo arquivo de rota e view, usando `res.send()` ao invés de `res.render()` — e explicar em um comentário no código a diferença entre os dois métodos;
- Criar, dentro da view home/index.ejs, links (`<a href="...">`) para todas as outras páginas do site.

8. Entrega

Disponibilizar como Atividade20 no GITHUB.

→ Seuusuario/TAPWM/Atividade20